DSU FINAL PRACTICAL

Solved Answers - Slips 1 to 25

Data Structures Using C

Simple practical-exam programs with quick oral revision

Compatibility note

The programs use standard C syntax and are suitable for Turbo C and most online C compilers. They avoid Turbo-C-only screen commands such as clrscr() and getch().

Contents

Slip 01 - Insert an element in a one-dimensional array
Slip 02 - Delete an element from a one-dimensional array
Slip 03 - Find minimum and maximum elements in an array
Slip 04 - Search a specific element in an array
Slip 05 - Search numbers using linear search
Slip 06 - Search numbers using binary search
Slip 07 - Search strings using linear search
Slip 08 - Search strings using binary search
Slip 09 - Sort numbers using bubble sort
Slip 10 - Sort strings using bubble sort
Slip 11 - Sort numbers using insertion sort
Slip 12 - Sort numbers using selection sort
Slip 13 - Implement stack using push, pop and display
Slip 14 - Implement linear queue using insert, delete and display
Slip 15 - Implement circular queue using insert, delete and display
Slip 16 - Multiply two numbers using recursion
Slip 17 - Create and display a singly linked list
Slip 18 - Create a list and insert a node at the beginning
Slip 19 - Create a list and insert a node in the middle
Slip 20 - Create a list and insert a node at the end
Slip 21 - Create a list and delete a node from the beginning
Slip 22 - Create a list and delete a node from the middle
Slip 23 - Create a list and delete a node from the end
Slip 24 - Create a singly linked list and search for a node
Slip 25 - Create a binary search tree and display preorder, inorder and postorder

SLIP 01

Aim: Insert an element in a one-dimensional array.

```
#include <stdio.h>

int main()
{
   int a[10], n, i, pos, value;

   printf("Enter number of elements (maximum 9): ");
   scanf("%d", &n);
   if(n < 0 || n >= 10)
   {
      printf("Invalid array size");
      return 0;
   }

   printf("Enter array elements:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &a[i]);

   printf("Enter value to insert: ");
   scanf("%d", &value);
   printf("Enter position (1 to %d): ", n + 1);
   scanf("%d", &pos);

   if(pos < 1 || pos > n + 1)
   {
      printf("Invalid position");
      return 0;
   }

   for(i = n; i >= pos; i--)
      a[i] = a[i - 1];

   a[pos - 1] = value;
   n++;

   printf("Array after insertion:\n");
   for(i = 0; i < n; i++)
      printf("%d ", a[i]);

   return 0;
}
```

SLIP 02

Aim: Delete an element from a one-dimensional array.

```
#include <stdio.h>

int main()
{
   int a[10], n, i, pos;

   printf("Enter number of elements (maximum 10): ");
   scanf("%d", &n);
   if(n < 1 || n > 10)
   {
      printf("Invalid array size");
      return 0;
   }

   printf("Enter array elements:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &a[i]);

   printf("Enter position to delete (1 to %d): ", n);
   scanf("%d", &pos);

   if(pos < 1 || pos > n)
   {
      printf("Invalid position");
      return 0;
   }

   for(i = pos - 1; i < n - 1; i++)
      a[i] = a[i + 1];
   n--;

   printf("Array after deletion:\n");
   for(i = 0; i < n; i++)
      printf("%d ", a[i]);

   return 0;
}
```

SLIP 03

Aim: Find minimum and maximum elements in an array.

```
#include <stdio.h>

int main()
{
   int a[10], n, i, min, max;

   printf("Enter number of elements (1 to 10): ");
   scanf("%d", &n);
   if(n < 1 || n > 10)
   {
      printf("Invalid array size");
      return 0;
   }

   printf("Enter array elements:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &a[i]);

   min = max = a[0];
   for(i = 1; i < n; i++)
   {
      if(a[i] < min)
         min = a[i];
      if(a[i] > max)
         max = a[i];
   }

   printf("Minimum element = %d\n", min);
   printf("Maximum element = %d", max);
   return 0;
}
```

SLIP 04

Aim: Search a specific element in an array.

```
#include <stdio.h>

int main()
{
   int a[10], n, i, key, found = 0;

   printf("Enter number of elements (1 to 10): ");
   scanf("%d", &n);
   if(n < 1 || n > 10)
   {
      printf("Invalid array size");
      return 0;
   }

   printf("Enter array elements:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &a[i]);

   printf("Enter element to search: ");
   scanf("%d", &key);

   for(i = 0; i < n; i++)
   {
      if(a[i] == key)
      {
         printf("Element found at position %d", i + 1);
         found = 1;
         break;
      }
   }

   if(found == 0)
      printf("Element not found");
   return 0;
}
```

SLIP 05

Aim: Search numbers using linear search.

```
#include <stdio.h>

int main()
{
   int a[10], n, i, key, found = 0;

   printf("Enter number of elements (1 to 10): ");
   scanf("%d", &n);
   if(n < 1 || n > 10)
   {
      printf("Invalid array size");
      return 0;
   }

   printf("Enter array elements:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &a[i]);

   printf("Enter number to search: ");
   scanf("%d", &key);

   for(i = 0; i < n; i++)
   {
      if(a[i] == key)
      {
         printf("Number found at position %d", i + 1);
         found = 1;
         break;
      }
   }

   if(found == 0)
      printf("Number not found");
   return 0;
}
```

SLIP 06

Aim: Search numbers using binary search.

Note: Binary search requires numbers in ascending order.

```
#include <stdio.h>

int main()
{
   int a[10], n, i, key;
   int beg, end, mid, found = 0;

   printf("Enter number of elements (1 to 10): ");
   scanf("%d", &n);
   if(n < 1 || n > 10)
   {
      printf("Invalid array size");
      return 0;
   }

   printf("Enter elements in ascending order:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &a[i]);

   printf("Enter number to search: ");
   scanf("%d", &key);

   beg = 0;
   end = n - 1;
   while(beg <= end)
   {
      mid = (beg + end) / 2;
      if(a[mid] == key)
      {
         printf("Number found at position %d", mid + 1);
         found = 1;
         break;
      }
      else if(key < a[mid])
         end = mid - 1;
      else
         beg = mid + 1;
   }

   if(found == 0)
      printf("Number not found");
   return 0;
}
```

SLIP 07

Aim: Search strings using linear search.

```
#include <stdio.h>
#include <string.h>

int main()
{
   char a[10][20], key[20];
   int n, i, found = 0;

   printf("Enter number of strings (1 to 10): ");
   scanf("%d", &n);
   if(n < 1 || n > 10)
   {
      printf("Invalid number of strings");
      return 0;
   }

   printf("Enter single-word strings:\n");
   for(i = 0; i < n; i++)
      scanf("%19s", a[i]);

   printf("Enter string to search: ");
   scanf("%19s", key);

   for(i = 0; i < n; i++)
   {
      if(strcmp(a[i], key) == 0)
      {
         printf("String found at position %d", i + 1);
         found = 1;
         break;
      }
   }

   if(found == 0)
      printf("String not found");
   return 0;
}
```

SLIP 08

Aim: Search strings using binary search.

Note: Binary search requires strings in alphabetical order.

```
#include <stdio.h>
#include <string.h>

int main()
{
   char a[10][20], key[20];
   int n, i, beg, end, mid, found = 0;

   printf("Enter number of strings (1 to 10): ");
   scanf("%d", &n);
   if(n < 1 || n > 10)
   {
      printf("Invalid number of strings");
      return 0;
   }

   printf("Enter strings in alphabetical order:\n");
   for(i = 0; i < n; i++)
      scanf("%19s", a[i]);

   printf("Enter string to search: ");
   scanf("%19s", key);

   beg = 0;
   end = n - 1;
   while(beg <= end)
   {
      mid = (beg + end) / 2;
      if(strcmp(a[mid], key) == 0)
      {
         printf("String found at position %d", mid + 1);
         found = 1;
         break;
      }
      else if(strcmp(a[mid], key) < 0)
         beg = mid + 1;
      else
         end = mid - 1;
   }

   if(found == 0)
      printf("String not found");
   return 0;
}
```

SLIP 09

Aim: Sort numbers using bubble sort.

```
#include <stdio.h>

int main()
{
   int a[10], n, i, j, temp;

   printf("Enter number of elements (1 to 10): ");
   scanf("%d", &n);
   if(n < 1 || n > 10)
   {
      printf("Invalid array size");
      return 0;
   }

   printf("Enter array elements:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &a[i]);

   for(i = 0; i < n - 1; i++)
   {
      for(j = 0; j < n - 1 - i; j++)
      {
         if(a[j] > a[j + 1])
         {
            temp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp;
         }
      }
   }

   printf("Sorted array:\n");
   for(i = 0; i < n; i++)
      printf("%d ", a[i]);
   return 0;
}
```

SLIP 10

Aim: Sort strings using bubble sort.

```
#include <stdio.h>
#include <string.h>

int main()
{
   char a[10][20], temp[20];
   int n, i, j;

   printf("Enter number of strings (1 to 10): ");
   scanf("%d", &n);
   if(n < 1 || n > 10)
   {
      printf("Invalid number of strings");
      return 0;
   }

   printf("Enter single-word strings:\n");
   for(i = 0; i < n; i++)
      scanf("%19s", a[i]);

   for(i = 0; i < n - 1; i++)
   {
      for(j = 0; j < n - 1 - i; j++)
      {
         if(strcmp(a[j], a[j + 1]) > 0)
         {
            strcpy(temp, a[j]);
            strcpy(a[j], a[j + 1]);
            strcpy(a[j + 1], temp);
         }
      }
   }

   printf("Strings in alphabetical order:\n");
   for(i = 0; i < n; i++)
      printf("%s\n", a[i]);
   return 0;
}
```

SLIP 11

Aim: Sort numbers using insertion sort.

```
#include <stdio.h>

int main()
{
   int a[10], n, i, j, key;

   printf("Enter number of elements (1 to 10): ");
   scanf("%d", &n);
   if(n < 1 || n > 10)
   {
      printf("Invalid array size");
      return 0;
   }

   printf("Enter array elements:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &a[i]);

   for(i = 1; i < n; i++)
   {
      key = a[i];
      j = i - 1;
      while(j >= 0 && a[j] > key)
      {
         a[j + 1] = a[j];
         j--;
      }
      a[j + 1] = key;
   }

   printf("Sorted array:\n");
   for(i = 0; i < n; i++)
      printf("%d ", a[i]);
   return 0;
}
```

SLIP 12

Aim: Sort numbers using selection sort.

```
#include <stdio.h>

int main()
{
   int a[10], n, i, j, min, temp;

   printf("Enter number of elements (1 to 10): ");
   scanf("%d", &n);
   if(n < 1 || n > 10)
   {
      printf("Invalid array size");
      return 0;
   }

   printf("Enter array elements:\n");
   for(i = 0; i < n; i++)
      scanf("%d", &a[i]);

   for(i = 0; i < n - 1; i++)
   {
      min = i;
      for(j = i + 1; j < n; j++)
      {
         if(a[j] < a[min])
            min = j;
      }
      temp = a[i];
      a[i] = a[min];
      a[min] = temp;
   }

   printf("Sorted array:\n");
   for(i = 0; i < n; i++)
      printf("%d ", a[i]);
   return 0;
}
```

SLIP 13

Aim: Implement stack using push, pop and display.

Oral: Stack follows LIFO (Last In, First Out).

```
#include <stdio.h>

#define MAX 10

int main()
{
   int stack[MAX], top = -1;
   int choice, value, i;

   while(1)
   {
      printf("\n1. Push\n2. Pop\n3. Display\n4. Exit\n");
      printf("Enter choice: ");
      scanf("%d", &choice);

      switch(choice)
      {
         case 1:
            if(top == MAX - 1)
               printf("Stack Overflow\n");
            else
            {
               printf("Enter value: ");
               scanf("%d", &value);
               top++;
               stack[top] = value;
               printf("Element pushed\n");
            }
            break;
         case 2:
            if(top == -1)
               printf("Stack Underflow\n");
            else
            {
               printf("Popped element = %d\n", stack[top]);
               top--;
            }
            break;
         case 3:
            if(top == -1)
               printf("Stack is empty\n");
            else
            {
               printf("Stack (top to bottom):\n");
               for(i = top; i >= 0; i--)
                  printf("%d\n", stack[i]);
            }
            break;
         case 4:
            return 0;
         default:
            printf("Invalid choice\n");
      }
   }
}
```

SLIP 14

Aim: Implement linear queue using insert, delete and display.

Oral: Queue follows FIFO (First In, First Out).

```
#include <stdio.h>

#define MAX 10

int main()
{
   int queue[MAX], front = -1, rear = -1;
   int choice, value, i;

   while(1)
   {
      printf("\n1. Insert\n2. Delete\n3. Display\n4. Exit\n");
      printf("Enter choice: ");
      scanf("%d", &choice);

      switch(choice)
      {
         case 1:
            if(rear == MAX - 1)
               printf("Queue Overflow\n");
            else
            {
               printf("Enter value: ");
               scanf("%d", &value);
               if(front == -1)
                  front = 0;
               rear++;
               queue[rear] = value;
               printf("Element inserted\n");
            }
            break;
         case 2:
            if(front == -1)
               printf("Queue Underflow\n");
            else
            {
               printf("Deleted element = %d\n", queue[front]);
               front++;
               if(front > rear)
               {
                  front = -1;
                  rear = -1;
               }
            }
            break;
         case 3:
            if(front == -1)
               printf("Queue is empty\n");
            else
            {
               printf("Queue elements: ");
               for(i = front; i <= rear; i++)
                  printf("%d ", queue[i]);
               printf("\n");
            }
            break;
         case 4:
            return 0;
         default:
            printf("Invalid choice\n");
      }
   }
}
```

SLIP 15

Aim: Implement circular queue using insert, delete and display.

Oral: The modulo operator lets the circular queue wrap around.

```
#include <stdio.h>

#define MAX 10

int main()
{
   int queue[MAX], front = -1, rear = -1;
   int choice, value, i;

   while(1)
   {
      printf("\n1. Insert\n2. Delete\n3. Display\n4. Exit\n");
      printf("Enter choice: ");
      scanf("%d", &choice);

      switch(choice)
      {
         case 1:
            if((rear + 1) % MAX == front)
               printf("Circular queue is full\n");
            else
            {
               printf("Enter value: ");
               scanf("%d", &value);
               if(front == -1)
               {
                  front = 0;
                  rear = 0;
               }
               else
                  rear = (rear + 1) % MAX;
               queue[rear] = value;
               printf("Element inserted\n");
            }
            break;
         case 2:
            if(front == -1)
               printf("Circular queue is empty\n");
            else
            {
               printf("Deleted element = %d\n", queue[front]);
               if(front == rear)
               {
                  front = -1;
                  rear = -1;
               }
               else
                  front = (front + 1) % MAX;
            }
            break;
         case 3:
            if(front == -1)
               printf("Circular queue is empty\n");
            else
            {
               printf("Queue elements: ");
               i = front;
               while(1)
               {
                  printf("%d ", queue[i]);
                  if(i == rear)
                     break;
                  i = (i + 1) % MAX;
               }
               printf("\n");
            }
            break;
         case 4:
            return 0;
         default:
            printf("Invalid choice\n");
      }
   }
}
```

SLIP 16

Aim: Multiply two numbers using recursion.

Oral: Recursion is a function calling itself; the base condition stops the calls.

```
#include <stdio.h>

int multiply(int a, int b)
{
   if(b == 0)
      return 0;
   else if(b > 0)
      return a + multiply(a, b - 1);
   else
      return multiply(a, b + 1) - a;
}

int main()
{
   int a, b;

   printf("Enter two integers: ");
   scanf("%d%d", &a, &b);

   printf("Multiplication = %d", multiply(a, b));
   return 0;
}
```

SLIP 17

Aim: Create and display a singly linked list.

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
   int data;
   struct node *next;
};

int main()
{
   struct node *head = NULL, *tail = NULL;
   struct node *newnode, *temp;
   int n, i, value;

   printf("Enter number of nodes: ");
   scanf("%d", &n);
   if(n < 0)
   {
      printf("Invalid number of nodes");
      return 0;
   }

   for(i = 0; i < n; i++)
   {
      newnode = (struct node *)malloc(sizeof(struct node));
      if(newnode == NULL)
      {
         printf("Memory allocation failed");
         return 0;
      }
      printf("Enter data: ");
      scanf("%d", &value);
      newnode->data = value;
      newnode->next = NULL;

      if(head == NULL)
         head = newnode;
      else
         tail->next = newnode;
      tail = newnode;
   }

   if(head == NULL)
      printf("List is empty");
   else
   {
      printf("Linked list: ");
      temp = head;
      while(temp != NULL)
      {
         printf("%d -> ", temp->data);
         temp = temp->next;
      }
      printf("NULL");
   }
   return 0;
}
```

SLIP 18

Aim: Create a list and insert a node at the beginning.

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
   int data;
   struct node *next;
};

int main()
{
   struct node *head = NULL, *tail = NULL;
   struct node *newnode, *temp;
   int n, i, value;

   printf("Enter number of nodes: ");
   scanf("%d", &n);
   if(n < 0)
      return 0;

   for(i = 0; i < n; i++)
   {
      newnode = (struct node *)malloc(sizeof(struct node));
      if(newnode == NULL)
         return 0;
      printf("Enter data: ");
      scanf("%d", &value);
      newnode->data = value;
      newnode->next = NULL;
      if(head == NULL)
         head = newnode;
      else
         tail->next = newnode;
      tail = newnode;
   }

   printf("Enter value to insert at beginning: ");
   scanf("%d", &value);
   newnode = (struct node *)malloc(sizeof(struct node));
   if(newnode == NULL)
      return 0;
   newnode->data = value;
   newnode->next = head;
   head = newnode;

   printf("List after insertion: ");
   temp = head;
   while(temp != NULL)
   {
      printf("%d -> ", temp->data);
      temp = temp->next;
   }
   printf("NULL");
   return 0;
}
```

SLIP 19

Aim: Create a list and insert a node in the middle.

Note: This program inserts at the middle position calculated from the original node count.

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
   int data;
   struct node *next;
};

int main()
{
   struct node *head = NULL, *tail = NULL;
   struct node *newnode, *temp;
   int n, i, value, pos;

   printf("Enter number of nodes: ");
   scanf("%d", &n);
   if(n < 0)
      return 0;

   for(i = 0; i < n; i++)
   {
      newnode = (struct node *)malloc(sizeof(struct node));
      if(newnode == NULL)
         return 0;
      printf("Enter data: ");
      scanf("%d", &value);
      newnode->data = value;
      newnode->next = NULL;
      if(head == NULL)
         head = newnode;
      else
         tail->next = newnode;
      tail = newnode;
   }

   printf("Enter value to insert in the middle: ");
   scanf("%d", &value);
   pos = n / 2 + 1; /* 1-based insertion position */
   newnode = (struct node *)malloc(sizeof(struct node));
   if(newnode == NULL)
      return 0;
   newnode->data = value;

   if(pos == 1)
   {
      newnode->next = head;
      head = newnode;
   }
   else
   {
      temp = head;
      for(i = 1; i < pos - 1; i++)
         temp = temp->next;
      newnode->next = temp->next;
      temp->next = newnode;
   }

   printf("List after insertion: ");
   temp = head;
   while(temp != NULL)
   {
      printf("%d -> ", temp->data);
      temp = temp->next;
   }
   printf("NULL");
   return 0;
}
```

SLIP 20

Aim: Create a list and insert a node at the end.

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
   int data;
   struct node *next;
};

int main()
{
   struct node *head = NULL, *tail = NULL;
   struct node *newnode, *temp;
   int n, i, value;

   printf("Enter number of nodes: ");
   scanf("%d", &n);
   if(n < 0)
      return 0;

   for(i = 0; i < n; i++)
   {
      newnode = (struct node *)malloc(sizeof(struct node));
      if(newnode == NULL)
         return 0;
      printf("Enter data: ");
      scanf("%d", &value);
      newnode->data = value;
      newnode->next = NULL;
      if(head == NULL)
         head = newnode;
      else
         tail->next = newnode;
      tail = newnode;
   }

   printf("Enter value to insert at end: ");
   scanf("%d", &value);
   newnode = (struct node *)malloc(sizeof(struct node));
   if(newnode == NULL)
      return 0;
   newnode->data = value;
   newnode->next = NULL;

   if(head == NULL)
      head = newnode;
   else
      tail->next = newnode;

   printf("List after insertion: ");
   temp = head;
   while(temp != NULL)
   {
      printf("%d -> ", temp->data);
      temp = temp->next;
   }
   printf("NULL");
   return 0;
}
```

SLIP 21

Aim: Create a list and delete a node from the beginning.

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
   int data;
   struct node *next;
};

int main()
{
   struct node *head = NULL, *tail = NULL;
   struct node *newnode, *temp;
   int n, i, value;

   printf("Enter number of nodes: ");
   scanf("%d", &n);
   if(n < 0)
      return 0;

   for(i = 0; i < n; i++)
   {
      newnode = (struct node *)malloc(sizeof(struct node));
      if(newnode == NULL)
         return 0;
      printf("Enter data: ");
      scanf("%d", &value);
      newnode->data = value;
      newnode->next = NULL;
      if(head == NULL)
         head = newnode;
      else
         tail->next = newnode;
      tail = newnode;
   }

   if(head == NULL)
      printf("List is empty");
   else
   {
      temp = head;
      head = head->next;
      printf("Deleted node = %d\n", temp->data);
      free(temp);

      printf("List after deletion: ");
      temp = head;
      while(temp != NULL)
      {
         printf("%d -> ", temp->data);
         temp = temp->next;
      }
      printf("NULL");
   }
   return 0;
}
```

SLIP 22

Aim: Create a list and delete a node from the middle.

Note: For an even number of nodes, this program deletes the left-middle node.

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
   int data;
   struct node *next;
};

int main()
{
   struct node *head = NULL, *tail = NULL;
   struct node *newnode, *temp, *del;
   int n, i, value, pos;

   printf("Enter number of nodes: ");
   scanf("%d", &n);
   if(n < 0)
      return 0;

   for(i = 0; i < n; i++)
   {
      newnode = (struct node *)malloc(sizeof(struct node));
      if(newnode == NULL)
         return 0;
      printf("Enter data: ");
      scanf("%d", &value);
      newnode->data = value;
      newnode->next = NULL;
      if(head == NULL)
         head = newnode;
      else
         tail->next = newnode;
      tail = newnode;
   }

   if(head == NULL)
      printf("List is empty");
   else
   {
      pos = (n + 1) / 2; /* middle position; left-middle if n is even */
      if(pos == 1)
      {
         del = head;
         head = head->next;
      }
      else
      {
         temp = head;
         for(i = 1; i < pos - 1; i++)
            temp = temp->next;
         del = temp->next;
         temp->next = del->next;
      }
      printf("Deleted middle node = %d\n", del->data);
      free(del);

      printf("List after deletion: ");
      temp = head;
      while(temp != NULL)
      {
         printf("%d -> ", temp->data);
         temp = temp->next;
      }
      printf("NULL");
   }
   return 0;
}
```

SLIP 23

Aim: Create a list and delete a node from the end.

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
   int data;
   struct node *next;
};

int main()
{
   struct node *head = NULL, *tail = NULL;
   struct node *newnode, *temp;
   int n, i, value;

   printf("Enter number of nodes: ");
   scanf("%d", &n);
   if(n < 0)
      return 0;

   for(i = 0; i < n; i++)
   {
      newnode = (struct node *)malloc(sizeof(struct node));
      if(newnode == NULL)
         return 0;
      printf("Enter data: ");
      scanf("%d", &value);
      newnode->data = value;
      newnode->next = NULL;
      if(head == NULL)
         head = newnode;
      else
         tail->next = newnode;
      tail = newnode;
   }

   if(head == NULL)
      printf("List is empty");
   else if(head->next == NULL)
   {
      printf("Deleted node = %d\n", head->data);
      free(head);
      head = NULL;
   }
   else
   {
      temp = head;
      while(temp->next->next != NULL)
         temp = temp->next;
      printf("Deleted node = %d\n", temp->next->data);
      free(temp->next);
      temp->next = NULL;
   }

   printf("List after deletion: ");
   temp = head;
   while(temp != NULL)
   {
      printf("%d -> ", temp->data);
      temp = temp->next;
   }
   printf("NULL");
   return 0;
}
```

SLIP 24

Aim: Create a singly linked list and search for a node.

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
   int data;
   struct node *next;
};

int main()
{
   struct node *head = NULL, *tail = NULL;
   struct node *newnode, *temp;
   int n, i, value, key, pos = 1, found = 0;

   printf("Enter number of nodes: ");
   scanf("%d", &n);
   if(n < 0)
      return 0;

   for(i = 0; i < n; i++)
   {
      newnode = (struct node *)malloc(sizeof(struct node));
      if(newnode == NULL)
         return 0;
      printf("Enter data: ");
      scanf("%d", &value);
      newnode->data = value;
      newnode->next = NULL;
      if(head == NULL)
         head = newnode;
      else
         tail->next = newnode;
      tail = newnode;
   }

   printf("Enter value to search: ");
   scanf("%d", &key);
   temp = head;
   while(temp != NULL)
   {
      if(temp->data == key)
      {
         printf("Node found at position %d", pos);
         found = 1;
         break;
      }
      temp = temp->next;
      pos++;
   }

   if(found == 0)
      printf("Node not found");
   return 0;
}
```

SLIP 25

Aim: Create a binary search tree and display preorder, inorder and postorder.

Oral: Preorder = Root-Left-Right; Inorder = Left-Root-Right; Postorder = Left-Right-Root.

```
#include <stdio.h>
#include <stdlib.h>

struct node
{
   int data;
   struct node *left;
   struct node *right;
};

struct node *insert(struct node *root, int value)
{
   if(root == NULL)
   {
      root = (struct node *)malloc(sizeof(struct node));
      if(root == NULL)
         return NULL;
      root->data = value;
      root->left = NULL;
      root->right = NULL;
   }
   else if(value < root->data)
      root->left = insert(root->left, value);
   else
      root->right = insert(root->right, value);
   return root;
}

void preorder(struct node *root)
{
   if(root != NULL)
   {
      printf("%d ", root->data);
      preorder(root->left);
      preorder(root->right);
   }
}

void inorder(struct node *root)
{
   if(root != NULL)
   {
      inorder(root->left);
      printf("%d ", root->data);
      inorder(root->right);
   }
}

void postorder(struct node *root)
{
   if(root != NULL)
   {
      postorder(root->left);
      postorder(root->right);
      printf("%d ", root->data);
   }
}

int main()
{
   struct node *root = NULL;
   int n, i, value;

   printf("Enter number of nodes: ");
   scanf("%d", &n);
   if(n < 0)
      return 0;

   for(i = 0; i < n; i++)
   {
      printf("Enter data: ");
      scanf("%d", &value);
      root = insert(root, value);
   }

   printf("Preorder: ");
   preorder(root);
   printf("\nInorder: ");
   inorder(root);
   printf("\nPostorder: ");
   postorder(root);
   return 0;
}
```

QUICK ORAL / VIVA REVISION

Review these short answers after practising the programs.

Array insertion: Shift elements to the right to create space, then place the new value at the required position.

Array deletion: Remove the value at the chosen position by shifting later elements one place to the left.

Minimum and maximum: Initialize both with the first element, then compare all remaining elements.

Linear search: Checks elements one by one; time complexity is O(n).

Binary search: Searches a sorted array by halving the search interval; time complexity is O(log n).

String operations: strcmp() compares strings; strcpy() copies strings. String comparisons are case-sensitive.

Bubble sort: Compares adjacent elements and swaps them when they are in the wrong order.

Insertion sort: Stores a key, shifts larger values to the right, and inserts the key into the sorted part.

Selection sort: Finds the smallest element in the unsorted part and swaps it into the next position.

Stack: Follows LIFO: Last In, First Out. Push inserts; pop removes the top element. Overflow means full; underflow means empty.

Linear queue: Follows FIFO: First In, First Out. Insert at rear and delete from front. A linear queue can waste freed spaces.

Circular queue: Reuses array positions by wrapping indexes with the modulo operator (%).

Recursion: A function calls itself. A base condition stops recursive calls.

Singly linked list: Each node contains data and a pointer to the next node. The last node points to NULL.

Linked-list insertion: At beginning: newnode->next = head; head = newnode. At end, link the last node to the new node.

Linked-list deletion: Change the link around the node to remove, then call free() on the removed node.

Binary search tree: For this program, smaller values go left and equal or larger values go right.

Tree traversals: Preorder: Root-Left-Right. Inorder: Left-Root-Right. Postorder: Left-Right-Root.

Dynamic memory: malloc() allocates memory at run time; free() releases allocated memory.

Before running a program

- Check that n is within the declared array size.
- For string searching and sorting, include string.h and use strcmp() to compare strings.
- Binary search only works on sorted data.
- For linked lists, allocate nodes with malloc() and release deleted nodes with free().
- For queue and stack menu programs, choose Exit to stop the loop.